2026年NOICA联盟-NOIP模拟赛（二十二）-DIV2

	债权	小猴摘果	环宝调度	染色结果
题目名称	债权	小猴摘果	环宝调度	染色结果
题目类型	传统型	传统型	传统型	传统型
目录	`debt`	`monkey`	`treasure`	`colors`
可执行文件名	`debt`	`monkey`	`treasure`	`colors`
输入文件名	`debt.in`	`monkey.in`	`treasure.in`	`colors.in`
输出文件名	`debt.out`	`monkey.out`	`treasure.out`	`colors.out`
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	3.0 秒
内存限制	256 MB	256 MB	512 MB	512 MB
测试点数目	3（子任务）	20	20（4 个捆绑包）	25
测试点是否等分	否	是	否	是

提交源程序文件名

对于 C++ 语言	`debt.cpp`	`monkey.cpp`	`treasure.cpp`	`colors.cpp`

编译选项

对于 C++ 语言	`-lm -O2 -std=c++14`

注意事项与提醒（请选手务必仔细阅读）

1. 输入输出文件名必须使用英文小写。
2. 选手提交的源程序必须存放在已建立好的、且带有样例文件和下发文件的文件夹中，文件夹名称与对应试题英文名一致。
3. 文件名（包括程序名和输入输出文件名）必须使用英文小写。
4. C++ 中函数 `main()` 的返回值类型必须是 `int`，值必须为 0。
5. 若无特殊说明，结果比较方式为忽略行末空格、文末回车后的全文比较。
6. 程序可使用的栈空间大小与该题内存空间限制一致。
7. 每道题目所提交的代码文件大小限制为 100 KB。
8. 若无特殊说明，输入文件与输出文件中同一行的相邻整数均使用一个空格分隔。
9. 输入文件中可能存在行末空格，请选手使用更完善的读入方式避免出错。
10. 直接复制题面中的多行样例，数据将带有行号，建议选手直接使用对应目录下的样例文件进行测试。

11. 请务必使用题面中规定的编译参数，保证你的程序在本机能够通过编译。此外不允许在程序中手动开启其他编译选项，一经发现，本题成绩以 0 分处理。
12. 对于因未遵守以上规则对成绩造成的影响，相关申诉不予受理。

债权（debt）

【题目描述】

天色已晚，小 Y 决定在一处村庄过夜。这座村庄中有 n 个人，他们之间存在 m 个欠债关系，例如 A 欠 B 五元钱，而每还款 1 元钱就会给村子带来 0.1 元的成本，村长很苦恼如何能够通过债权转让的方式让成本最小化，于是求助于小 Y，现在小 Y 已经帮助村长完成了债权的转让，只需要你计算出最小化后的成本即可。

债权转让：若 A 欠 B 五元钱，B 欠 C 五元钱，可以通过债权转让变为 A 欠 C 五元钱。

【输入格式】

从文件 `debt.in` 中读入数据。

输入文件的第一行包含两个正整数 n, m，以空格隔开，表示一共有 n 个人和 m 个欠债关系。

接下来的 m 行每行包含三个正整数 x, y, z，以空格隔开，表示 x 欠 y 的钱数为 z。

【输出格式】

输出到文件 `debt.out` 中。

输出一个保留一位的小数，表示最小化后的成本。

【样例 1 输入】

```
5 3
1 2 5
2 3 3
4 5 4
```

【样例 1 输出】

```
0.9
```

【样例 1 解释】

1 欠 2 五元钱、2 欠 3 三元钱可以转变为 1 欠 2 两元钱、1 欠 3 三元钱，然后可以得到一共需要还款 9 元钱，成本为 0.9 元钱。

【样例 2 输入】

```
2 2
1 2 5
2 1 5
```

【样例 2 输出】

```
0.0
```

【样例 2 解释】

1 和 2 债务相抵后互不相欠，成本为 0.0 元。

【数据范围】

对于 100% 的数据，$1 \le n \le 100000$，$1 \le m \le 200000$，$1 \le x, y \le n$，$1 \le z \le 10^9$。

子任务	分值	n	m
1	30	$n \le 10$	$m \le 20$
2	30	$n \le 100$	$m \le 10000$
3	40	无特殊限制	无特殊限制

小猴摘果（monkey）

【题目背景】

在一片神秘的果园中，生活着一只爱吃果子的聪明小猴子。果园的树上挂满了果子，这些果子按照固定的顺序排成一排，从左到右编号为 1 到 n。每个果子的大小各不相同。小猴子按照果子的顺序依次走过，但它可以选择摘下某些果子，也可以不摘。

【题目描述】

然而，果园的守护精灵有一个特殊的要求：**小猴子摘取的所有果子必须满足以下条件，才能被视为合法采摘**：

1. **摘下的任意三个果子中（假设编号为 x, y, z，且满足 $x < y < z$）**，不能出现这样的大小关系：**编号最小的果子 < 编号最大的果子 < 中间编号的果子。**

   （例如，第 x 个果子是最小的，第 y 个果子是最大的，第 z 个果子是中等大小的，这样的采摘组合是不允许的！）

2. 同样的，**摘下的任意三个果子中（假设编号为 x, y, z，且满足 $x < y < z$）**，不能出现这样的大小关系：**中间编号的果子 < 编号最大的果子 < 编号最小的果子。**

   （例如，第 y 个果子是最小的，第 x 个果子是最大的，第 z 个果子是中等大小的，这样的采摘组合也不允许！）

果园里共有 n 个果子，小猴子希望知道：**在这些规则限制下，小猴子能选择的所有合法摘果子方式的数量。**

1. 小猴子至少要摘取一个果子。
2. 所有的摘果方式需要保证摘取果子的顺序和果子编号顺序一致。
3. 结果需要对 998244353 取模。

一共有 T 组数据，对每组数据输出一个答案。

【输入格式】

从文件 `monkey.in` 中读入数据。

- 第一行包含一个非负整数 T，表示测试数据组数。
- 每组数据第一行包含一个整数 n（$1 \leq n \leq 10^5$），表示果子的数量。
- 每组数据第二行包含 n 个整数 $p_1, p_2, \ldots, p_n$（$1 \leq p_i \leq n$，互不相同），表示果子从左到右的大小。

【输出格式】

输出到文件 `monkey.out` 中。

- 每组数据输出一行，表示所有合法摘果方式的数量，对 998244353 取模。

【样例 1 输入】

```
1
3
2 1 3
```

【样例 1 输出】

```
7
```

【样例 2 输入】

```
1
4
1 3 2 4
```

【样例 2 输出】

```
13
```

【样例 1 解释】

对于输入的果子大小序列 `[2, 1, 3]`，其所有非空子序列中满足规则的摘果方式如下：

- `[2]`
- `[1]`
- `[3]`
- `[2, 1]`
- `[2, 3]`
- `[1, 3]`
- `[2, 1, 3]`

总共有 7 种合法摘果方式。

【数据范围】

对于所有测试点，保证 $T \leq 20$，$n \leq 10^5$。保证 p 是一个排列。

测试点编号	特殊性质
1	$T = 0$
$2, 3, 4$	$n \leq 15$
$5, 6, 7$	$n \leq 18$
$8, 9, 10$	$n \leq 100$
$11, 12, 13, 14$	$n \leq 2000$
$15, 16, 17$	$n \leq 3 \times 10^4$
$18, 19, 20$	无特殊限制

【提示】

注意：本题输入文件较大，您可以使用这里提供的输入优化，或者使用自己的输入优化：

```
int rd(){
   char s=getchar();int v=0;
   while(s<'0'||s>'9')s=getchar();
   while(s>='0'&&s<='9')v=v*10+s-'0',s=getchar();
   return v;
}
```

环宝调度（treasure）

【题目描述】

有 n 个宝藏，编号为 $1, 2, \ldots, n$。第 i 个宝藏有一个整数价值 a_i，并且有一个指向位置 b_i。

你需要恰好操作每个宝藏一次。操作一个当前编号为 u 的宝藏时：

1. 将宝藏 u 当前的价值加入总收益；
2. 如果 $b_u \neq 0$，则把宝藏 u 当前的价值加到宝藏 b_u 的价值上。
3. 操作后，宝藏 u 将被看作已经操作过，不能再次操作。

总收益是所有宝藏都被操作后获得的价值之和。

请你求出所有合法操作顺序中的最大总收益。

【输入格式】

从文件 `treasure.in` 中读入数据。

第一行一个整数 n。

第二行包含 n 个整数 $a_1, a_2, \ldots, a_n$，表示每个宝藏的初始价值。

接下来 n 行，第 i 行包含一个整数 b_i，表示宝藏 i 的指向位置。$b_i = 0$ 表示没有指向位置，否则保证 $1 \le b_i \le n$。

【输出格式】

输出到文件 `treasure.out` 中。

输出一个整数，表示最大总收益。

【样例 1 输入】

```
2
1 2
2
1
```

【样例 1 输出】

```
5
```

【样例 1 解释】

一种最优顺序是先操作宝藏 2，再操作宝藏 1。第一次操作获得 2，并把 2 加到宝藏 1，于是宝藏 1 的当前价值变成 3，第二次操作获得 3，总收益为 5。

【样例 2 输入】

```
8
5 -2 4 -7 3 1 -1 2
2
3
4
1
6
7
8
5
```

【样例 2 输出】

```
33
```

【样例 3】

见选手目录下的 `treasure/treasure3.in` 与 `treasure/treasure3.ans`。

【数据范围与子任务】

所有数据满足 $1 \le n \le 2 \times 10^5$，$-10^6 \le a_i \le 10^6$，且 $0 \le b_i \le n$。

子任务	分值	n	特殊性质
1 ~ 3	15	≤ 8	无
4 ~ 11	30	$\leq 2 \times 10^5$	保证指向位置 b_i 不存在环
12 ~ 15	25	$\leq 2 \times 10^5$	$a_i \leq 0$
16 ~ 20	30	$\leq 2 \times 10^5$	无

每个捆绑包中的所有测试点均通过后，才能获得对应分值。

染色结果（colors）

【题目描述】

给定一棵有 n 个顶点的树，顶点编号为 1 至 n。最初所有顶点都没有颜色。

选择一个 1 至 n 的排列，按照这个排列的顺序，依次为每个顶点染色。处理顶点 u 时，令 S 为与 u 相邻且已经染色的顶点的颜色集合，将 u 的颜色设为 $\operatorname{mex}(S)$。

这里，$\operatorname{mex}(S)$ 表示没有出现在集合 S 中的最小非负整数。例如，$\operatorname{mex}(\varnothing) = 0$，$\operatorname{mex}(\{0, 2\}) = 1$。

完成染色后，按顶点编号得到颜色数组 $(c_1, c_2, \ldots, c_n)$。求所有排列能够得到的**不同颜色数组**的数量，答案对 998244353 取模。

不同排列如果得到相同的颜色数组，只计算一次。顶点编号固定，不会因为树具有对称性而把两个不同数组视为相同。

【输入格式】

从文件 `colors.in` 中读入数据。

第一行包含一个正整数 n。

接下来 $n - 1$ 行，每行包含两个整数 u, v，表示顶点 u, v 之间有一条边。

【输出格式】

输出到文件 `colors.out` 中。

输出一行一个整数，表示不同颜色数组的数量对 998244353 取模的结果。

【样例 1 输入】

```
4
1 2
2 3
3 4
```

【样例 1 输出】

```
4
```

【样例 1 解释】

所有可能的颜色数组为 $(0,1,0,1)$、$(1,0,1,0)$、$(0,1,2,0)$ 和 $(0,2,1,0)$，共 4 种。

例如，处理顺序 $(1,2,3,4)$ 和 $(1,3,2,4)$ 都得到 $(0,1,0,1)$，只计算一次；顺序 $(1,4,2,3)$ 得到 $(0,1,2,0)$。

【样例 2 输入】

```
5
1 2
1 3
1 4
1 5
```

【样例 2 输出】

```
2
```

【样例 2 解释】

该样例是一棵星形树。两种结果分别为 $(0,1,1,1,1)$ 和 $(1,0,0,0,0)$。

该样例满足测试点 11 的约束条件。

【样例 3 输入】

```
8
1 2
1 3
3 4
1 5
5 6
5 7
7 8
```

【样例 3 输出】

```
16
```

【样例 3 解释】

这棵树可以产生颜色 3。例如按 $(2,4,6,8,3,7,5,1)$ 处理，得到颜色数组 $(3,0,1,0,2,0,1,0)$。输出仍需统计所有不同颜色数组。

该样例满足测试点 $1,2$ 的约束条件。

【样例 4】

见选手目录下的 `colors/colors4.in` 与 `colors/colors4.ans`。该样例满足测试点 $3,4$ 的约束条件。

【样例 5】

见选手目录下的 `colors/colors5.in` 与 `colors/colors5.ans`。该样例满足测试点 $5, 6$ 的约束条件。

【样例 6】

见选手目录下的 `colors/colors6.in` 与 `colors/colors6.ans`。该样例满足测试点 15 的约束条件。

【样例 7】

见选手目录下的 `colors/colors7.in` 与 `colors/colors7.ans`。该样例满足测试点 17 的约束条件。

【样例 8】

见选手目录下的 `colors/colors8.in` 与 `colors/colors8.ans`。该样例满足测试点 $20 \sim 25$ 的约束条件。

【数据范围】

对于所有测试数据，$1 \leq n \leq 2 \times 10^5$，$1 \leq u, v \leq n$，输入的边构成一棵树。

每个测试点独立计分，分值均为 4 分。表中"无限制"表示仅受上述整体范围约束。

测试点编号	$n \leq$	$\Delta \leq$	特殊性质
$1, 2$	8	无限制	无
$3, 4$	12	无限制	无
$5, 6$	2000	无限制	无
$7, 8$	5000	无限制	无
9	5000	无限制	A
10	2×10^5	无限制	A
11	2×10^5	无限制	B
12	5000	无限制	C
13	2×10^5	无限制	C
14	5000	3	无
15	2×10^5	3	无
16	5000	6	无
17	2×10^5	6	无
$18, 19$	30000	无限制	无
$20 \sim 25$	2×10^5	无限制	无

记 Δ 为树中顶点度数的最大值，顶点的度数指与它相邻的顶点数量。

特殊性质 A：树是一条链。

特殊性质 B：存在一个顶点与其余所有顶点相邻。

特殊性质 C：同时删除原树中所有度数为 1 的顶点后，剩余部分是一条链、一个顶点或空图。这样的树也称为毛毛虫树。